



VISION

To be recognized for excellent engineering, developing global leaders both in educational and research in the domain of computer science and wireless engineering.

MISSION

1. To create self-learning environment by facilitating leadership qualities, team spirit and ethical responsibilities.
2. To improve department-industry collaboration, interaction with professional society through technical knowledge and internship program.
3. To promote research and development with current techniques through well qualified resources in the area of computer science and wireless engineering.

Practical No. 08

Aim: Implement Naïve Bayes Classification in Python

Software Required: Python IDE, Anaconda, VS Code, Jupyter Nootbook, Google Colab.

Theory:

Naive Bayes Classifiers

Naive Bayes is a machine learning classification algorithm that predicts the category of a data point using probability. It assumes that all features are independent of each other. Naive Bayes performs well in many real-world applications such as spam filtering, document categorization and sentiment analysis.

It is named as "Naive" because it assumes the presence of one feature does not affect other features. The "Bayes" part of the name refers to its basis in Bayes' Theorem.

The main idea behind the Naive Bayes classifier is to use Bayes' Theorem to classify data based on the probabilities of different classes given the features of the data. It is used mostly in high-dimensional text classification

- The Naive Bayes Classifier is a simple probabilistic classifier and it has very few number of parameters which are used to build the ML models that can predict at a faster speed than other classification algorithms.
- It is a probabilistic classifier because it assumes that one feature in the model is independent of existence of another feature. In other words, each feature contributes to the predictions with no relation between each other.
- Naive Bayes Algorithm is used in spam filtration, Sentimental analysis, classifying articles and many more.

Bayes' Theorem provides a principled way to reverse conditional probabilities. It is defined as:

$$P(y|X) = \frac{P(X|y) \cdot P(y)}{P(X)}$$

Where:

- $P(y|X)$: Posterior probability, probability of class y given features X
- $P(X|y)$: Likelihood, probability of features X given class y
- $P(y)$: Prior probability of class y
- $P(X)$: Marginal likelihood or evidence

Naive Bayes Working

1. Terminology

Consider a classification problem (like predicting if someone plays golf based on weather). Then:

y is the class label (e.g. "Yes" or "No" for playing golf)

$X=(x_1,x_2,...,x_n)$ is the feature vector (e.g. Outlook, Temperature, Humidity, Wind)

A sample row from the dataset:

$X=(\text{Rainy}, \text{Hot}, \text{High}, \text{False}), y=\text{No}$

This represents:

What is the probability that someone will not play golf given that the weather is Rainy, Hot, High humidity, and No wind?

2. The Naive Assumption

The "naive" in Naive Bayes comes from the assumption that all features are independent given the class. That is:

$$P(x_1, x_2, \dots, x_n | y) = P(x_1 | y) \cdot P(x_2 | y) \cdots P(x_n | y)$$

Thus, Bayes' theorem becomes:

$$P(y | x_1, \dots, x_n) = \frac{P(y) \cdot \prod_{i=1}^n P(x_i | y)}{P(x_1)P(x_2) \dots P(x_n)}$$

Since the denominator is constant for a given input, we can write:

$$P(y | x_1, \dots, x_n) \propto P(y) \cdot \prod_{i=1}^n P(x_i | y)$$

3. Constructing the Naive Bayes Classifier

We compute the posterior for each class y and choose the class with the highest probability:

$$\hat{y} = \arg \max_y P(y) \cdot \prod_{i=1}^n P(x_i | y)$$

This becomes our Naive Bayes classifier.

Applications of Naive Bayes Classifier

- **Spam Email Filtering:** Classifies emails as spam or non-spam based on features.
- **Text Classification:** Used in sentiment analysis, document categorization, and topic classification.
- **Medical Diagnosis:** Helps in predicting the likelihood of a disease based on symptoms.
- **Credit Scoring:** Evaluates creditworthiness of individuals for loan approval.
- **Weather Prediction:** Classifies weather conditions based on various factors.

Advantages of Naive Bayes Classifier

- Easy to implement and computationally efficient.
- Effective in cases with a large number of features.
- Performs well even with limited training data.
- It performs well in the presence of categorical features.
- For numerical features data is assumed to come from normal distributions

Disadvantages of Naive Bayes Classifier

- Assumes that features are independent, which may not always hold in real-world data.
- Can be influenced by irrelevant attributes.
- May assign zero probability to unseen events, leading to poor generalization.

Program

```
import pandas as pd

# Dataset
data = {
    'Outlook':
    ['Rainy','Rainy','Overcast','Sunny','Sunny','Sunny','Overcast','Rainy','Rainy','Sunny','Rainy','Overcast','Overcast','Sunny'],
    'Temperature':
    ['Hot','Hot','Hot','Mild','Cool','Cool','Cool','Mild','Cool','Mild','Mild','Mild','Hot','Mild'],
    'Humidity':
    ['High','High','High','High','Normal','Normal','Normal','High','Normal','Normal','Normal','Normal','High','Normal','High'],
    'Windy':
    [False,True,False,False,False,True,True,False,False,False,True,True,False,True],
    'Play Golf': ['No','No','Yes','Yes','Yes','No','Yes','No','Yes','Yes','Yes','Yes','Yes','Yes','No']
}

df = pd.DataFrame(data)
print(df)
from collections import Counter

# Separate by class
yes_df = df[df['Play Golf']=='Yes']
no_df = df[df['Play Golf']=='No']

# Prior probabilities
```

```

p_yes = len(yes_df) / len(df)
p_no = len(no_df) / len(df)

print("Prior P(Yes)=", p_yes)
print("Prior P(No)=", p_no)

# Prediction query
query = {'Outlook':'Sunny', 'Humidity':'High', 'Windy':True}

# Likelihood calculation
def likelihood(df_class, query):
    prob = 1.0
    for feature, value in query.items():
        count = sum(df_class[feature]==value)
        prob *= count / len(df_class)
    return prob

likelihood_yes = likelihood(yes_df, query)
likelihood_no = likelihood(no_df, query)

# Posterior using Bayes theorem
posterior_yes = likelihood_yes * p_yes
posterior_no = likelihood_no * p_no

print("\nLikelihood P(X|Yes)=", likelihood_yes)
print("Likelihood P(X|No)=", likelihood_no)
print("Posterior P(Yes|X)=", posterior_yes)
print("Posterior P(No|X)=", posterior_no)

prediction = "Yes" if posterior_yes > posterior_no else "No"
print("\n Prediction: Play Golf =", prediction)

```

Output:

Outlook Temperature Humidity Windy Play Golf

0	Rainy	Hot	High	False	No
1	Rainy	Hot	High	True	No
2	Overcast	Hot	High	False	Yes
3	Sunny	Mild	High	False	Yes
4	Sunny	Cool	Normal	False	Yes
5	Sunny	Cool	Normal	True	No
6	Overcast	Cool	Normal	True	Yes
7	Rainy	Mild	High	False	No
8	Rainy	Cool	Normal	False	Yes
9	Sunny	Mild	Normal	False	Yes
10	Rainy	Mild	Normal	True	Yes
11	Overcast	Mild	High	True	Yes
12	Overcast	Hot	Normal	False	Yes
13	Sunny	Mild	High	True	No

Prior P(Yes)= 0.6428571428571429

Prior P(No)= 0.35714285714285715

Likelihood P(X|Yes)= 0.037037037037037035

Likelihood P(X|No)= 0.19200000000000003

Posterior P(Yes|X)= 0.02380952380952381

Posterior $P(\text{No}|\text{X}) = 0.06857142857142859$

Prediction: Play Golf = No

Conclusion: Naïve Bayes helps understand to Naïve Bayes is a fast and simple probabilistic classifier that works well on small datasets and text data, but its assumption of feature independence can limit accuracy when features are correlated.